The Chase

Self-scrolling writing for the pen-pal site. Build spec for coding agents. Phone, mouse, and full screen share one mechanism. Includes write mode.

1. What it is

The chase is a follow camera for handwriting. The writer keeps making letters in about the same spot on the screen, and the page slides under them so the writing runs across the whole line. It feels like always moving forward without ever scrolling.

On a phone you are zoomed in, focused on one letter at a time, and only see the edges of the letters next to it. The thumb works a writing area on the right side of the screen, roughly 2 inches across. On a full-size screen with a mouse, the view is wider, so the page only starts to move when the pointer gets close to the right side. The mechanism is the same in both. Only the size of the chase zone differs.

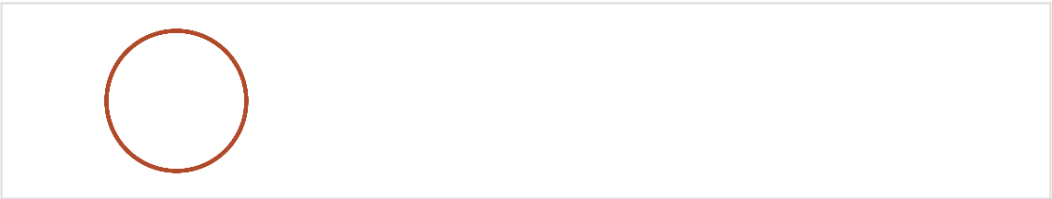
2. The helix (read this before writing any code)

Cursive is a helix going across the page. Split it into two parts:

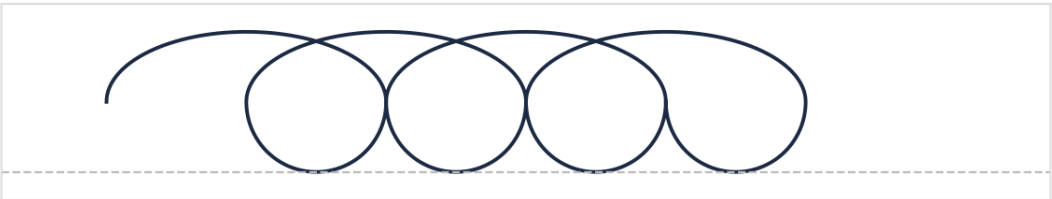
- **The line** is time. It moves forward, left to right.
- **The circle** is the movement of the letters. Cursive is mostly a circular motion.

The writer's thumb (or mouse) only does the circle. The chase supplies the line. Put them together and you get the helix on the page. The figure below was produced by running the exact rule in section 5 on a thumb that does nothing but circle in one spot.

What the thumb does on the glass: circles in one spot



What lands on the page, rate 1.0: the circle rides the line (the helix)



Same thumb motion, rate 0.5: tighter, more overlapping loops



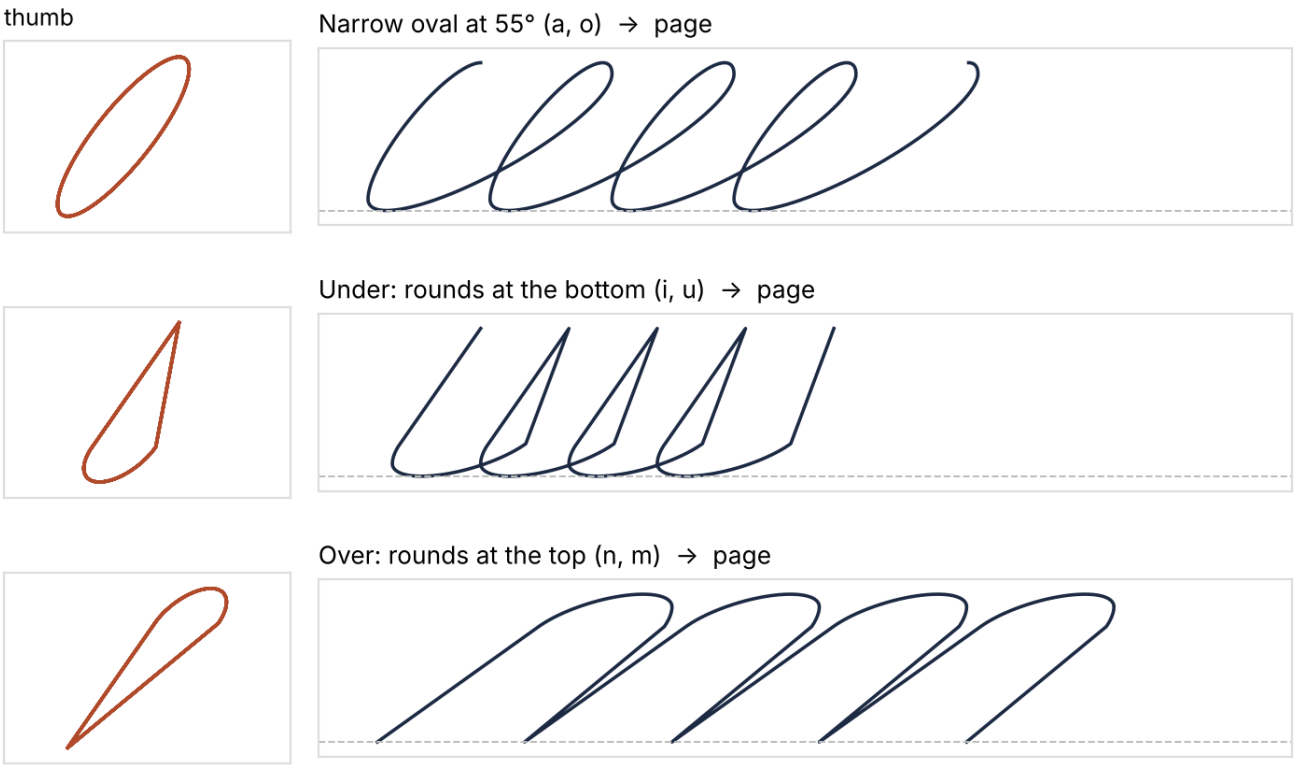
3. Slant, ovals, over and under

Copperplate rides at about a 55° slant. And when you actually make the letters, the motion isn't a circle. It's a narrow oval, tilted at that slant. It's always ovals, but sometimes the stroke goes under and sometimes over: an **i** rounds at the bottom, an **n** rounds at the top, and an **a** or **o** is the full oval.

The rule in section 5 needs no change for any of this. It only reads the rightward part of each movement, so it handles every shape the same way:

- **Going down the slant** (the main stroke) the pen moves left, so the page holds still. That stroke lands at exactly the angle the writer made it.
- **Coming back up** (the thin connecting stroke) the pen moves right, so the page flows with it. That stroke lands stretched forward, longer and flatter.
- **Under** (i, u): the page holds on the way down, then flows through the bottom round and up the connector.
- **Over** (n, m): the page flows up the connector and over the top, then holds while the pen comes down the stem.

So the slant on the page is the writer's own slant. The chase never tilts, straightens, or corrects anything. Do not add slant correction or rotate ink. The figure below runs the same section 5 rule on a narrow 55° oval and on under and over turns (shapes are sketches; the rule is exact).



Slant guides (optional): faint guide lines at the slant angle can be drawn on the page. Draw them in *page* coordinates so they slide with the page, which means the writer always sees a fresh slant line right where their thumb is working. The angle is a setting (`SLANT_DEG`, starting at 55). `RATE` changes how flat the connecting upstrokes land; it never changes the angle of the downstrokes.

4. The one rule: it moves when they act

The page moves only when the writer acts. The second they move right, it moves. If they move more, it moves more. It moves with their rate. It is a flow.

There is no perfect speed to find, because there is no speed at all. Nothing in this mechanism measures time, estimates velocity, smooths motion, or runs on a timer. The only input is the writer's own movement, event by event. That is why it always matches their rate exactly: it *is* their rate.

- Thumb or mouse moves **right** (inside the chase zone): the page slides at that same rate.
- Writer **pauses**: the page does not move.
- Writer moves **left**: the page does not move.
- Writer lifts the pen: the page does not move.

5. The math

Two coordinate spaces. **Screen** = where the pointer is on the glass, in screen pixels. **Page** = where the ink lives on the letter, in page units. The camera offset `cam` connects them:

```
page_x = screen_x / zoom + cam        page_y = screen_y / zoom + cam_y
```

The whole chase is one line, run once per pointer movement event:

```
dx = screen_x_now - max(screen_x_prev, zone_start)
if dx > 0:
    cam += (dx / zoom) * RATE
```

RATE = 1.0 matches the line to the thumb exactly. The `max(..., zone_start)` makes sure only the part of a movement that happens inside the chase zone counts, so a stroke that starts in the free area and crosses into the zone only pushes the page by the part past the boundary.

Why a circle becomes the helix: over one full circle the thumb ends where it started, so it has moved right by the circle's width and left by the same amount. The page only counts the rightward half, so after each circle the page has advanced by one circle width. On the page, each circle is stretched into a forward loop. A slow small circle makes a tight letter; a wide sweep right makes a long connector.

6. Exact order inside the movement handler

Less careful builds get this wrong. Do it in this order, on every pointer movement:

```
on pointerdown(e):
    prev_x = e.x_in_canvas            # sets the starting point; never moves the camera
    start a new stroke
    stroke.cam_before = { cam, cam_y, line }   # saved for undo (section 10)

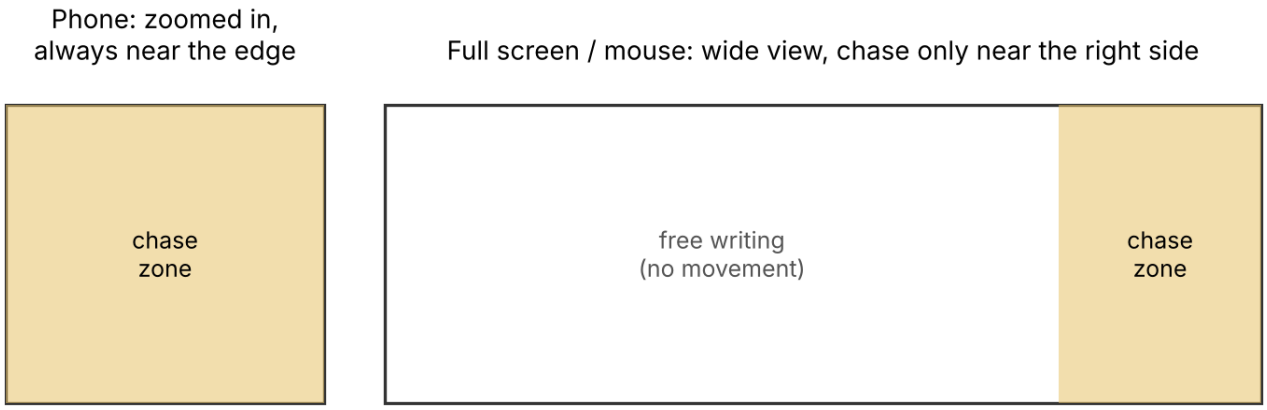
on pointermove(e):
    for each point in e.getCoalescedEvents() (or just e if unsupported):
        x = point.x_in_canvas         # SCREEN coordinate, raw from the event
        dx = x - max(prev_x, zone_start)
        if dx > 0:
            cam += (dx / zoom) * RATE  # 1. move the camera first
        page_point = toPage(x, point.y)  # 2. then convert using the NEW cam
        stroke.add(page_point, pressure, ...)  # 3. then store the ink
        prev_x = x                    # 4. last

on pointerup / pointercancel:
    end the stroke; camera does not move
```

```
render (requestAnimationFrame is fine for drawing):
    draw the page translated by -cam; never change cam in here
```

The render loop only *draws*. It never changes `cam`. All camera movement comes from the movement handler.

7. The chase zone: phone and full screen



`zone_start` is a screen x position. Left of it, the writer writes freely and nothing moves. Right of it, the rule in section 5 applies.

- **Phone:** the view is zoomed in to about one letter, so the writer is basically always near the edge. The zone covers the writing area on the right side of the screen.
- **Full screen / mouse:** the view is wide, so there is a lot of room to write in the left and middle. The page only starts to move when the pointer gets close to the right side.
- Same function, same rule. Only `zone_start` changes with the view width.

8. Zoom: the middle ground

On phone there is a minor ability to zoom in and out, clamped to a range. It never gets too small, because you can't make letters small, and never too big, because then you can't see the stuff around the letter. There is a middle ground where it just feels nice to write the letters. Implement zoom as a clamped value (`ZOOM_MIN`, `ZOOM_MAX`) and always divide screen movement by the current zoom in the math above, so the chase keeps matching the thumb at any zoom.

9. The existing line reset

The site already has a feature: when you get to the very right edge, it resets to the next line on the left side. Keep it. The chase moves the camera along the current line; the line reset still takes over when the end of the line is reached. When the reset fires, the camera jumps to the start of the next line, and `prev_x` is set to the current pointer position so the jump itself never counts as a rightward movement.

10. Write mode (monk mode)

Writing happens in its own mode, on both sizes. First you choose your pen settings, then you enter write mode, and then you're just in it. On mobile, pretty much the whole screen is the writing area, so it feels immersive, almost like an app. It's a monk mode: nothing on screen but the writing, so you can focus.

Write mode has only three buttons:

- **Escape**: leave write mode.

- **Undo**: remove the last line you made **and put the camera back where it was before you made it.**
- **Play**: watch it written out. Play is also the *done* button: after playback it moves straight to the next part (save the letter, send it to a friend, and so on), so you never have to press escape and then go down to find it.

How undo works with the chase. Every stroke saves the camera at the moment the pen went down (`cam_before` in section 6). Undo removes the last stroke and restores that saved camera exactly. Do not try to reverse-compute the camera from the stroke. Just restore what was saved. This also brings back the line if the stroke came right after a line reset. After undo, `prev_x` is cleared, so the next pen-down starts fresh.

```
undo():
    if draft.strokes is empty: return
    stroke = draft.strokes.pop()
    cam, cam_y, line = stroke.cam_before   # exact restore, no math
    prev_x = none
    redraw
```

Undo only works on the draft. Ink is append-only once it lands in the book, so the strokes in write mode are a draft until you press Play/done. Only then is the finished letter or writing appended to the ink chain.

11. What gets stored

Ink is always stored in **page** coordinates, never screen coordinates. The camera is a view, not part of the letter. This keeps the stroke data identical whether it was written on a phone with the chase, on a desktop with no chase, or uploaded from a photo, so playback and the profile file are unaffected by how it was written.

12. Mistakes to avoid

- **No time.** No timers, no setInterval, no velocity, no easing, no lerp, no "smooth follow" animation. If a variable has "speed", "velocity", "dt" or "ms" in its name, the build is wrong.
- **Compute dx from screen coordinates only.** Never from page coordinates. Page x already includes cam, so using it feeds the camera back into itself and the page runs away.
- **Never move the camera on pointerdown.** Putting the pen down somewhere new is not an act of moving right.
- **Never move the camera left.** cam only increases during a line (the line reset is the only exception).
- **Use coalesced events** where available so fast strokes don't skip points and the page tracks every bit of motion.
- **Don't move the camera in the render loop.** Render draws; the movement handler moves.
- **Same code for phone and desktop.** Do not write two chase systems.
- **Undo restores the saved camera.** Never back-calculate the camera from the removed stroke, and never append draft strokes to the ink chain before Play/done.

13. Test checklist

- Circle in one spot on the phone: the page shows forward loops like the figures in sections 2 and 3.
- Write a narrow oval at 55°: downstrokes on the page keep exactly that angle; only the upstrokes stretch forward.
- Write i u i u (under) and n m n m (over): the page holds on every downstroke.
- Hold the pen still mid-stroke: the page does not move at all.
- Move left only: the page does not move.
- Move right fast, then slow: the page moves fast, then slow, exactly with the hand.

- Desktop: writing in the left and middle never moves the page; only the right side does.
- Change zoom, then circle again: loops keep the same shape relative to the letter size.
- Reach the end of the line: the existing reset still goes to the next line on the left, with no extra jump.
- Replay a written letter: playback looks identical to the ink, with no trace of camera movement.
- Write three lines, undo three times: the camera returns to exactly where each line began, including across a line reset.
- Press Play: playback runs, then it lands on the save / send step with no escape needed.
- Escape mid-letter, come back: nothing was appended to the ink chain.

14. Values Tabor sets after testing

These are knobs, not decisions. Ship them as constants in one place so they can be felt out on the phone:

- `RATE`: start at 1.0 (matches the thumb exactly). Lower makes tighter, more overlapping loops.
- `zone_start` for phone and for full screen.
- `SLANT_DEG` for the optional slant guides (start at 55), and whether guides are on by default.
- `ZOOM_MIN` / `ZOOM_MAX` and the default zoom on phone.
- Which edge triggers the existing line reset when the chase is active (the page's line end or the view's edge).